

「全部任せる」の一言からウェブゲーム公開まで Claude Code の ultracode モードによるマルチエージェント開発の 事例報告

貝淵蒼馬

概要

紐で吊ったボールをクレーンで振って壁の向こうに止める研究コード（軌道最適化と運動方程式）をもとに、ブラウザゲーム「ゆらしてピタッ」を作り、Cloudflare で公開した。依頼は「この振り子の遊びで web ゲームアプリ作りたい。全てそっちに任せるから、最強に面白いゲームを作って。」の 2 文だけで、設計・実装・検証をすべて Claude Code に任せた。本稿は、その開発をセッションの記録（会話ログ、ワークフローの台本、サブエージェント 70 体の実行ログ）から再構成し、どのようなプロンプトでどのような工程を踏んだのか、完成度を上げるために何をしたのかを報告する。開発の本体は約 14.4 時間の 1 セッションで、7 本のワークフローが延べ 70 体のサブエージェントを動かし、ツール呼び出しは 5,921 回、記録上のトークンは約 1,912 万だった。工程は「複数案の生成と審査による設計」「契約と仮実装の先行」「ファイル所有権を分けた並列実装と独立レビュー」「人間らしいボットによる難易度調査」「粗探し・反証・修正の 3 段の総点検」の順に進んだ。人が入れた指示は公開までに約 10 回で、内容は公開先の指定、ヒントの提供、試遊の感想、公開操作に限られる。試遊の感想「難しすぎる」は、エージェントの審査でランキング対象外の補助に下げられた「ゆれ止め自動制御」を、既定でオンの機能として復活させた。公開日（約 12 時間）の API 要求は 23,505 件（Cloudflare の分析値）で、クリアの記録が届いた端末は少なくとも 2,342 台だった。

1 はじめに

筆者はこれまで、MathWorks のブログ記事 [1] にある「クレーンで吊り荷を振り上げて壁を越えさせる」問題を Python で再現し、2 枚の壁の間の狭い隙間にボールを出し入れする問題へ拡張した [2]。軌道は直接多重シューティング法と内点法（CasADi/IPOPT）で求めている。

今回はこれを遊べる形にするため、Claude Code に次の依頼だけを出した。

この振り子の遊びで web ゲームアプリ作りたい。全てそっちに任せるから、最強に面白いゲームを作って。

依頼をあえて大まかにしたのは、権限をすべて渡したときに何ができあがるかを調べたかったからである。Claude Code の実行モードは `/effort` で ultracode にした。このモードでは、Claude がワークフロー（複数のサブエージェントを JavaScript の台本で動かす仕組み）を自分で書き、実質的な作業ごとに走らせる。

できあがったゲームは公開直後から予想以上に遊ばれた（6 節）。本稿の目的は、その開発過程を記録から再構成し、次の 3 点を明らかにすることである。

1. Claude（以下、指揮役）がサブエージェントにどのようなプロンプトを投げたか。
2. 開発がどのような工程で進んだか。
3. 完成度を上げるためにどのような工夫をしたか。人は何をしたか。

■資料 分析には次の記録を使った。すべて `~/.claude/projects/` の下に保存されたものである。

- 開発セッション（2026-09-22 13:47 ~ 09-23 04:12 JST, Opus 5）の会話ログ
- 7 本のワークフローの台本・結果・進行記録（実行時の台本をそのまま保存したもの）
- サブエージェント 70 体それぞれの実行ログ

- 公開セッション（09-23 09:43 ～ 12:34, Opus 5.5）と、公開後の集計セッションの会話ログ

本稿で引用するプロンプトは台本からの抜粋で、英語の原文のままである。なお、本稿の分析と執筆も Claude Code が行った。自己評価に偏りうる点は 7.3 節で述べる。

2 対象と条件

2.1 題材

研究コードの物理モデルは、質量 2 kg の台車と、長さ 1 m の紐で吊った 1 kg のボールからなる。入力には台車に掛ける水平の力（最大 40 N）だけである。高さ 0.75 m の壁を越えるには、ボールを約 64° 振り上げる必要がある。最適化器は、壁から 2 cm 離れ、紐の張力を 1.5 N 以上に保つ最短時間の軌道を 1 問あたり約 30 秒で解く。

2.2 人が入れた指示

表 1 に、公開までに人が入れた指示をすべて示す。指揮役への指示は約 10 回で、ゲームの中身に関わるものは、試遊の感想（H7）と表示言語の既定（H8）だけである。

表 1: 人が入れた指示（時刻は JST）

	時刻	内容
H1	09-22 13:47	/effort ultracode のうえで最初の依頼（上記の 2 文）
H2	13:56	「共有ランキングとかについて、cloudflare で公開する予定だから、無料枠の範囲内で D1 とか割り」と自由に使って」
H3	17:52	「現状の web ui とか軽く見てみたいから、npm run dev のやり方とか教えて」
H4	09-23 02:07	「CasADi 3.8.1 が js/WebAssembly に対応しているから、必要に応じて使ってね」
H5	02:22	「Opus 5.5 がリリースされたから、キリがいたら再起動して Opus 5.5 で続けて」
H6	09:43	（再起動後）「game/docs/HANDOFF.md を読んで続きをやって」
H7	10:55	試遊の感想「難しすぎる。減衰させようとする」と加速する。矢印キーだけだと微調整がしづらすぎる。…操作難易度を下げる工夫をしてほしい」
H8	11:20 頃	「日本語をデフォルトに」（作業中の割り込み。指揮役はキューに積み、作業後に処理した）
H9	11:42～11:59	公開の手順を尋ね、日替わりの起点日を選択肢から選び、Cloudflare の画面で D1 と Worker を作成して ID を渡した
H10	12:15～12:34	宣伝動画の作成、コミットと push, GitHub Release の依頼

2.3 実行環境

開発セッションのモデルは Opus 5（1M context）で、全サブエージェントも同じモデルだった。公開セッションは H5 の依頼どおり Opus 5.5 に切り替えた。機械は 24 コアの Linux 1 台で、ブラウザテストには Playwright の Chromium・Firefox・WebKit を使った。

3 工程の全体像

開発セッションの流れを図 1 に、ワークフローごとの規模を表 2 に示す。指揮役自身のツール呼び出しは 63 回だけで、残りの 5,921 回はサブエージェントが行った。指揮役は、台本を書き、結果を読み、次の台本に「決定事項」を書き込む役に徹していた。

表 2: ワークフローごとの規模. 時間は壁時計, トークンは実行記録に残る合計

ワークフロー (台本名)	エージェント	時間 [分]	ツール呼び出し	トークン [万]
- pendulum-game-design (中止)	5	6.5	29	39
① pendulum-game-design	10	183.0	330	208
② yurapita-foundation	3	44.5	240	84
③ yurapita-modules	19	134.9	2,351	784
④ yurapita-integrate-polish	9	241.7	1,399	332
⑤ yurapita-balance-review	22	238.1	1,211	404
⑥ casadi-wasm-spike	2	114.6	361	59
合計	70		5,921	1,912

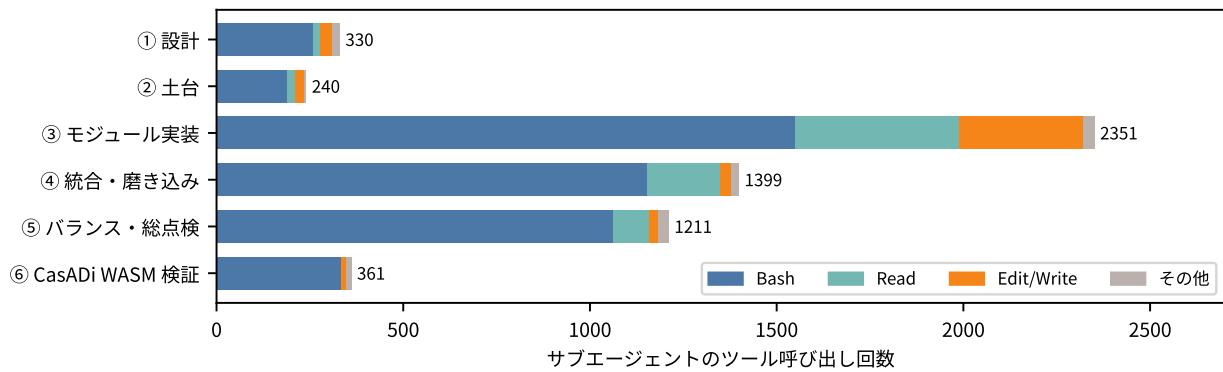


図 2: ワークフローごとのツール呼び出しの内訳. ほとんどが Bash (テスト, ビルド, シミュレーション, スクリーンショット撮影) で, 「読んで書く」より「動かして確かめる」に手数を使っている.

工程は次の順に進んだ. 各段の詳細は 4 節で述べる.

- ① **設計**: 5 つの方向からゲーム案を出し, 3 人の審査役が採点し, 1 つの設計書にまとめ, 抜けを直す.
- ② **土台**: npm プロジェクト, モジュール間の型 (契約), 全ファイルの仮実装, 面データ, 仮の AI ゴーストを作る.
- ③ **並列実装**: 9 つのモジュールを担当エージェントが同時に作り, それぞれ別のエージェントがレビューして直す.
- ④ **統合と磨き込み**: モジュール間の依頼を片付け, 人間らしいボットで難易度を調べ, 全体を通して確かめる.
- ⑤ **バランス調整と公開前の総点検**: 難易度を直し, 4 つの観点で粗探し→反証→修正を行う.
- ⑥ **実現性の検証**: 人のヒント (H4) を受けて, CasADi の WebAssembly 版がブラウザで使えるかを調べる.

これに続いて, モデルの切り替えのための引き継ぎ, 人の試遊を受けた調整, 公開, 宣伝動画の作成があった (5 節).



図 1: 開発セッションにおけるサブエージェントの稼働時間。1 行が 1 体。点線 U1~U5 は人の発言 (表 1 の H1~H5)。灰色の短い行は、H2 を受けて中止した最初の設計ワークフロー。

4 各工程とプロンプト

4.1 ① 設計：独立した 5 案，3 つの観点の審査，統合，批評

指揮役は最初に研究コードの README と物理モジュールを読み，Node とブラウザとコア数を確かめてから，設計ワークフローを書いた．全エージェントに共通の前置き CONTEXT には，運動方程式，壁の寸法，最適化器の性能，公開の条件がすべて数値で書かれている（リスト 1）．

リスト 1: 全エージェント共通の前置き（抜粋）

```
It simulates a gantry crane: a trolley (mass M=2.0 kg) on a horizontal rail at height rail_y=1.25 m, a ball
(m=1.0 kg, radius 0.06 m) hanging on a massless inextensible STRING of length L=1.0 m. Input = horizontal force F
on the trolley (|F|<=40 N) ...
(M+m) x'' + m L cos(th) th'' - m L w^2 sin(th) = F - b v
...
- Cheating: a public leaderboard will be attacked. Consider server-side verification: the Worker re-simulates a
submitted per-tick input log with the SAME TypeScript physics module ... This needs bit-exact cross-engine
determinism (fixed timestep, inputs quantized per tick, own sin/cos implemented with + - * / only because
Math.sin differs between V8/JavaScriptCore/SpiderMonkey) and must fit the 10 ms CPU budget ...
- Scope: built by a team of AI coding agents in one session ..., so prefer depth and polish over sprawling
feature lists. Must be genuinely fun, not a tech demo.
```

最初の設計ワークフローは，公開先を claude.ai の Artifact と想定していた．走り始めて 9 分後に H2 (Cloudflare で公開，D1 を使ってよい) が届いたため，指揮役はワークフローを止め，前置きの公開条件を Cloudflare の無料枠（1 日 10 万要求，1 要求 10 ms の CPU，D1 の読み書きの上限）とサーバー側の再シミュレーションによる不正対策に書き換えて，やり直した．リスト 1 の後半はこのとき足された文である．

■5 つの案 5 体のエージェントに，それぞれ別の方向からゲーム案を 1 つずつ作らせた（表 3）．方向の指示は 1～2 文で，参考作品の名前を挙げて具体的にしている．たとえば mastery には “*The headline hook is ‘can a human beat the optimal-control AI?’*”，mobile には “*The first 10 seconds must be self-explanatory without text*” と指示した．出力は JSON スキーマで固定し，面の寸法をメートル単位で 12～20 面ぶん書かせた．案を作るエージェントの多くは，研究コードの最適化器や自作のシミュレーションを実際に動かして，面が解けるかを確かめていた（arcade 案のエージェントは 96 分，38 回の Bash を使った）．

表 3: 5 つの案と審査の結果．点数は 3 人の審査役の重み付き合計（楽しさ ×2，最初の 1 分 ×1.5，作りやすさ ×1.5，独自性，研究への忠実さ）

方向	案の名前	タグライン（抜粋）	面数	合計点
mastery	ゆらしてピタッ	最適制御 AI に，人間の指で勝て．	19	169.8
mobile	フリピタ	振って，越えて，ピタッと止める．	16	158.0
arcade	スレスレ・クレーン	壁まで，あと 3 ミリ．	18	152.0
campaign	ユラピタ ー振り子クレーン ー	ゆらして，越えて，ぴたっと止める．	18	137.0
sandbox	ふりこパーキング	押せるのは台車だけ．	20	128.8

■3 つの観点の審査 審査役は「任天堂やインディーの感覚をもつ試遊責任者（楽しさ）」「three.js，WebAudio，Cloudflare，決定的な物理を担う主任エンジニア（作りやすさ）」「X でリンクを見てスマホで開き，30 秒しか我慢しない日本の技術者（プレイヤー）」の 3 人である．3 人とも mastery を土台に選んだ．それぞれが他の案から「移植すべき案」と「捨てるべき案」を挙げた．移植されたのは，縦画面の下半分を操作盤にする相対ドラッグ（arcade），Wordle 風の共有文付きの「今日の 5 球」（mobile），自動生成の表示名（mobile），壁との隙間を mm で見せる演出と「論文ライン」20 mm（arcade），振り子の折り返しで振れ幅に応じた音程の拍子木が鳴る音（campaign）などである．一方で，紐が壁の角に巻きつく物理や面エディタ（sandbox），点数・コンボの経済（arcade），そして指を離すと LQR の自動制御が揺れを止める仕組み（mobile）を中心に据えることは，3 人がそろって退けた．理由は「題名の動詞である『ピタッ』をプレイヤーから取り上げる」「『人間の指で最適制御 AI

に勝て』と矛盾する」で、残すとしてもランキング対象外の補助に留める、という判断だった。この項目は5節で、人の試遊によって形を変えて復活する。

■統合と批評 統合役には「別々の AI エンジニアが互いに話さずに並行して作れるほど正確に」書くよう求め、12 章の必須項目（物理仕様、面の表、データ型、API と SQL、無料枠の計算、モジュールごとの所有ファイル、受け入れ基準付きのマイルストーン）を列挙した。できた設計書 `GAME_DESIGN.md` は約 2,300 行になった。

続く批評役には、“*the lead engineer who must start building tomorrow*” の立場で、矛盾や解けない面を探し、設計書を直接直すよう指示した。批評役は 53 分・177 回のツール呼び出しを使い、最適化器と自作のサーボ・シミュレーションを実際に走らせて、次のような問題を見つけて直した。

- 3 面の制限時間が再現できなかった（1-3 の 3.0 秒は 13 分たっても解けず、4-1 の 4.0 秒は実行不能）。確認できた値に直した。
- 「一度ダッシュすると 64° に届く」という演出の前提は、紐のたるみを入れた本来のモデルでは 61.9° だった。
- 送信のまとめ方が D1 無料枠の「1 回 50 文まで」と「本文 16 KB まで」を超えていた。
- 長い走行はサーバーの検証予算を常に超え、ランキングに永久に載らなかった。
- 人間が AI に勝てる余地（最適解より何 % 速く解けるか）を各面で測る検査を追加した。

4.2 ② 土台：契約と仮実装を先に作る

設計書を読んだ指揮役は、いきなり並列実装に入らず、2 体を並べて土台を作らせた。1 体目は npm プロジェクト一式、設定、テストの枠組み（Node, 3 ブラウザ, Cloudflare の workerd, E2E）と、**全ソースファイルの型つき仮実装**を作った。中身は `throw new Error('not implemented')` でよいが、プロジェクト全体が型チェックを通る状態にする。また、物理モジュールで `Math.sin` を使うと lint とテストが落ちることを、実際に置いて確かめるよう指示している。2 体目は Python 側で 18 面のデータ、研究コードと照合するためのテスト用データ、それに**二分探索を省いた仮の AI ゴースト**（M2a）を作った。本番のゴーストは 1~2 時間かかるので、他の担当を待たせないためである。最後に 3 体目が、報告を鵜呑みにせず全コマンドを自分で実行して検証した。

土台が終わると、指揮役は本番の AI ゴースト計算（全面で最短時間を二分探索）を 12 コアで裏に流し、その間に並列実装を始めた。計算は 84 分で終わり、たとえば 3-1 「非力モーター」の AI のタイムは仮の 13.9 秒から 5.57 秒になった。

4.3 ③ 並列実装：ファイル所有権と独立レビュー

実装は、物理コア、ゲーム進行、入力、描画、音、UI、保存と通信、Worker と D1、日替わりの面の生成の 9 モジュールに分け、担当 O1~O9 に同時に作らせた。全員が同じ作業ツリーで動くので、共通の規則を細かく決めている（リスト 2）。

リスト 2: 並列実装の共通規則（抜粋）

```
- Only create/modify files you own (listed in your task). Never edit other owners' files, never edit GAME_DESIGN.md ..., package.json, lockfile or shared configs. If you need a change elsewhere, write it in your report (requests_for_other_owners).
- Never run git commands that change state. Never run npm install. Do not touch anything outside game/.
- Other owners are implementing their stubs at the same time: code strictly against the contract types; when you need another module's behaviour in tests, use small fakes. ...
- If you run Playwright, ALWAYS set unique ports: PW_PREVIEW_PORT and PW_DEV_PORT env vars ...
- Quality bar: this must become a genuinely fun, polished game (Nintendo-level feel), not a tech demo. Verify your work by actually running it (tests, builds, screenshots you look at with the Read tool). Report failures honestly.
```

各担当のプロンプトには、所有ファイルの一覧、設計書の該当節、受け入れ基準が書かれている。描画担当には“*LOOK at your screenshots with the Read tool and iterate until the scene is genuinely attractive and legible on a phone*”, UI 担当には“*Avoid generic 'AI-looking' UI (no purple gradients, no glassmorphism)*” と、見

た目の基準まで指定した。物理担当には、研究コードとの誤差 (1e-9)、3 ブラウザと workerd でのビット単位的一致、5400 ステップを 5 ms 以内で計算できることを求めた。

台本は依存関係をそのまま表している (リスト 3)。各モジュールは作り終わると、**物理コアの完成を待って**から、別のエージェントのレビューに回る。レビュー役は元の担当と同じファイルを所有し、“*Adversarially review*”して直す。

リスト 3: 作る→レビューする、の依存つきパイプライン (台本の抜粋)

```
const o1Build = agent(COMMON + ... + o1.task, { label: `build:${o1.key}`, phase: 'Build', schema: REPORT })
const o1Done = o1Build.then(r => r, () => null)
const reviewOf = async (m, build) => {
  await o1Done // real integration with src/sim is possible only after the physics core is done
  return agent(COMMON + REVIEW_TASK(m, build), { label: `review:${m.key}`, phase: 'Review', schema: REPORT })
}
```

レビューは形だけでは終わらなかった。表 4 に主な発見を示す。いずれも作った本人が「受け入れ基準はすべて通過」と報告したあとに見つかったものである。

表 4: 独立レビューで見つかった主な不具合 (③)

モジュール	発見 (重大度)
物理	レール端で台車が止まっているとき、ボールが振れの途中で空中に止まる (高)。「GC ゼロ」の主張が誤りで、最適化後のコードは 1 ステップ 32 バイトを確保していた (中)
ゲーム進行	振れ幅メーターなど、ボールに付いて動く表示が本番の画面に一つも出ていなかった。スマホを回すと目標位置が消えた (高)
描画	NaN が 1 回入ると時計とカメラが壊れ、次の開始まで画面が真っ白になる (高)
音	失敗を知らせる衝突音が、振り子の拍子音より 6~9 dB 小さく、しかもスマホのスピーカーで鳴らない低域に偏っていた (高)
UI	NaN が「NaN mm」とそのまま表示された。練習走行にも「自己ベスト更新」が出た (高)
保存と通信	2 つ目のタブを閉じると、もう一方のタブで保存した進行が消える。3 ブラウザで再現 (高)
Worker	仕様の誤りはないが、テストが報告より弱く、分の境目をまたぐと失敗する不安定なテストがあった (中)

4.4 ④ 統合と磨き込み：決定事項、難易度調査、通しプレイ

③の結果を読んだ指揮役は、自分で開発サーバーを立ててスクリーンショットを 2 枚見たうえで、各担当の「他の担当への依頼」と「設計からの逸脱」を裁定し、**決定事項 D1~D10** として次の台本の冒頭に書いた (“*LEAD DECISIONS (binding; they override the spec where they differ)*”). た例えば D10 は、縦画面の上 4 割が空白の紙になっていることを指摘し、カメラの追従幅とスコア表示の配置を担当 2 人に分けて直させた。

同じワークフローで、**人間らしい操作をするボット**による難易度調査も並行させた。指示は具体的で、タッチは「折れ点 6 個以下、間隔 0.15 秒以上、目標速度 5m/s 以下の折れ線」、キーボードは「切り替え 7 回以下、間隔 0.12 秒以上」とし、最良の操作に時刻 $\sigma = 40$ ms, 位置 $\sigma = 1$ cm のぶれを加えて 50 回ずつ成功率を測るよう求めた。エージェントは交差エントロピー法で操作を最適化し、本物のシミュレーター・サーボ・入力処理を使って 18 面と日替わり 12 面を調べた。結果、4-3「たまご」は銅メダルが取れない、5-2 は銅メダルでも 9%、キーボードでは 2-2 を一度もクリアできない、日替わりの AI のタイムは甘すぎる、といったことがわかった。

最後に統合役が全ファイルの所有権を引き継ぎ、全テストを通したうえで、390×844 のタッチと 1280×720 のマウスで、実際の操作による通しプレイを撮影して確かめた。スクリーンショットから、結果カードの上のボタンが押せない、320 px 幅でボタンの文字が 1 文字ずつ折れる、など 10 件を見つけて直した。

4.5 ⑤ バランス調整と公開前の総点検：粗探し→反証→修正

指揮役は難易度調査の結果から、5-2 の奥の壁を 5 cm ずらす、たまごの割れる張力を 16 N から 18 N に上げる、キーボードを「短く押すと 5 cm 動く」ようにする、日替わり 84 面の AI のタイムを 0.1 秒刻みで解き直す、と決めた。これと並行して、公開前の総点検を次の 3 段で行った（リスト 4）。

1. **粗探し**：サーバー（攻撃者と急な人気集中の視点）、物理（再生と不正対策の一致）、ゲーム進行、見た目と音の 4 観点で、読み取り専用のエージェントが不具合を探す。「具体的な失敗の筋書きがないものは書くな」と指示した。
2. **反証**：中・高の指摘 1 件ごとに、別のエージェントが反証を試みる。“*Default to real=false if you cannot substantiate it*”。
3. **修正**：観点ごとに 1 体が、確定した指摘を直し、1 件ごとに回帰テストを足す。

リスト 4: 反証役へのプロンプト（全文）

```
# Role: skeptical verifier (read-only on the game source; scratch under /tmp/claude-1000/yp-verify/${f.id}/).
Try to REFUTE this finding. Read the code paths involved, and if cheap, reproduce it with a small script/test.
Default to real=false if you cannot substantiate it. If real, give the right severity and the best fix.
Finding: ${JSON.stringify(f)}
```

指摘は 26 件（高 3、中 9、低 14）出た。中・高の 12 件はすべて反証に耐えたが、高とされた 3 件はすべて「中」に下げられた。主なものを表 5 に示す。修正役は、回帰テストが元のコードで本当に落ちることも確かめている（サーバーの回帰テスト 20 件のうち 16 件が元のコードで失敗した）。

表 5: 公開前の総点検で確定した指摘（抜粋）

観点	指摘
サーバー	他人の公開リプレイを、新しい秘密鍵で何度でも送り直してランキング上位 100 を埋められる
サーバー	IPv6 では同じ /64 から無制限に送れる（回数制限のキーが完全なアドレス）
サーバー	別サイトからの単純な POST (text/plain) を受け付けてしまう
進行	日替わりの 5 球目の最中に画面を隠すと、その球の結果が送られない
進行	日替わりの起点日が公開日より後のままだと、再訪した人の日替わりが止まる
見た目	王冠や自己ベストの帯が結果カードの裏に描かれ、見えない
見た目	320～375 px の画面で、結果ボタンの文字が単語の途中で折れたり 1 文字に切れたりする

最後のゲート役は、全テスト（単体 1,299、ブラウザ 80、E2E 262）を通し、難易度を測り直した。たまごの銅メダル到達率は 0% から 32% に、5-2 は 9% から 35% になった。ただしキーボードで 2-2 をクリアできる率は 0.2% から 1.0% にしか上がらず、ゲート役は「キーボードの失敗の 8～9 割は最後の止め方ではなく壁への衝突」と原因を正しく報告している。この弱点は、人の試遊で指摘されるまで残った。

4.6 ⑥ 実現性の検証：CasADi WebAssembly

⑤の最中に、人が「CasADi 3.8.1 が js/WebAssembly に対応しているから、必要に応じて使ってね」と伝えた (H4)。指揮役はすぐに組み込まず、ゲームのファイルに触れない場所で試す 2 体のワークフローを立てた。1 体目は研究の最適化問題を JS に移し、Python と目的関数がビット単位で一致すること、ブラウザの Web Worker で動き、既存の解から始めれば 0.4～0.5 秒で解き直せることを示した。2 体目はその数値を測り直すとともに、配布されている IPOPT のプラグインに METIS 4 の関数が 39 個含まれ、そのライセンスが無許可の再配布を禁じていることを見つけた。指揮役はこの結果から「いまは使わない。METIS を外した版か、定式化を変えて fatrop を使えるようになったら、失敗した瞬間から AI が解き直す機能を入れる」と決め、引き継ぎメモに残した。

4.7 引き継ぎ：モデルの切り替え

人が「キリがいたら Opus 5.5 で続けて」と依頼した (H5) とき、裏で 2 本のワークフローが動いていた。指揮役はこの依頼をタスクキューに積み、両方が終わるのを待ってから、現状、決定事項、残作業、公開手順、注意点をまとめた `game/docs/HANDOFF.md` を書き、自分の記憶ファイルからもそこを指すようにしたうえで、再起動を依頼した。新しいセッションは「HANDOFF.md を読んで続きをやって」の一言で作業を再開し、残っていた細かい表示の不具合を直した。

5 人の試遊と公開

5.1 「難しすぎる」

公開前に筆者が遊び、「難しすぎる。減衰させようとする」と加速する。矢印キーだけだと微調整がしづらすぎる」と伝えた (H7)。Opus 5.5 の指揮役はワークフローを使わず、自分で原因を調べて直した。キーボードは「速度を操る」方式で、ちょっと押しただけで 0.17 秒で最高速に達するため、止めようとして押すたびに揺れを大きくしていた。ボットの調査でもキーボードは 2-2 のクリア率が 1% 未満だった。

指揮役は、ゴールの 30 cm 以内で手を離すと離散時間 LQR が揺れを止めてボールをゴールの内側へ寄せる「ゆれ止めアシスト」を実装した。ゲイン候補の格子探索と閉ループのシミュレーションを本物の物理で行い、止まるまでの時間はパッド面で 15~24 秒から 2~3.6 秒に、ポケット面で 7~15 秒から 1.6~1.9 秒に縮んだ。記録されるのは台車に掛けた力の列だけなので、サーバーの再シミュレーションによる検証はそのまま使える。あわせてキーボードの加速を半分にし、それに合わせて入力テスト 26 件の期待値を直す作業だけを、「テストを骨抜きにしない」という条件つきでサブエージェント 1 体に任せた。

この変更は、①の審査で 3 人がそろって退けた「指を離すと LQR が揺れを止める」案 (mobile 案) とほぼ同じものである。審査役は『『ピタッ』をプレイヤーから取り上げる』として、ランキング対象外の補助に留めるよう求め、設計書でも優先度の最も低い P2 に置かれていた。人が実際に遊ぶと、それがないと難しすぎた。結果として、この補助は既定でオン、ランキング対象で出荷された。

5.2 公開

公開の段階で、指揮役は外部に影響する操作を人に委ねた。日替わりの起点日 (公開後は変えられない) を選択肢で尋ね、ビルドがまっさらな状態から通ることを確かめたうえでコミットと push を行い、Cloudflare の画面での D1 の作成と GitHub リポジトリの接続は、手順を示して人にやってもらった。これは筆者が「自分でリポジトリを選ぶところをやってみたい」と頼んだためでもある。公開は 2026-09-23 11:59 JST だった。

その後、宣伝動画を依頼した (H10)。指揮役はゲームのテスト用フックを使って本物のタッチ操作を録画し、シミュレーターで「ちゃんとクリアできる指の動かし方」を先に探してから撮影した。ナレーションは VOICEVOX (ずんだもん)、編集は Remotion で、27.6 秒の縦動画になった。



図 3: 完成したゲームの画面（宣伝動画用に実際の操作で録画した映像から切り出し）。下半分が指で操作する盤，上の数字が AI と自分のタイム。（d）の下のグラフは AI が掛けた力 $F(t)$ 。

6 成果物の規模と反響

6.1 規模

表 6 に，公開時点のコードの規模を示す。設計段階の前置きでは「全体で 5～15k 行でよい」と書いていたが，テストを含めて約 5.5 万行になった。そのうち約 1.9 万行がテストである。

表 6: 公開時点のコードの行数（game/ 以下）

部分	行数	部分	行数
UI (src/ui)	10,323	音 (src/audio)	1,871
ゲーム進行 (src/core)	4,563	入力 (src/input)	1,615
描画 (src/render)	4,286	Worker	1,565
Python ツール (tools)	3,327	保存・通信・共有	1,810
開発用デモ (dev)	2,825	テスト (tests)	18,793
物理 (src/sim)	2,430	設計書 (GAME_DESIGN.md)	2,337

中身は次のとおりである。

- 5 ワールド 18 面。全面に，研究の最適化器で解いた AI のゴーストとパー（AI のタイム）がある。
- 最適化器で解いた日替わり用の 84 面と，Wordle 風の共有文つきの「今日の 5 球」。
- 物理は研究コードと同じ運動方程式に，紐のたるみ，張り直しの撃力，レール端の衝突を足したもの。sin/cos を自前で実装し，Chromium・Firefox・WebKit・Cloudflare workerd で結果がビット単位で一致する。
- Cloudflare Worker と D1 による面ごとの世界ランキング。送られた入力列をサーバーが同じ物理で再計算して検証する。
- 効果音はすべて WebAudio の合成，テクスチャはすべてプログラムで生成し，外部の素材ファイルはない。

6.2 反響

公開日（2026-09-23 JST）の全体が終わった直後（09-24 00:24 JST）に、Cloudflare の分析 API（GraphQL）から Worker と静的ファイル配信の要求数を、本番の D1 から端末数を、いずれも読み取り専用で取得した（表 7）。公開は 11:59 なので、公開日の数値は約 12 時間ぶんである。

表 7: 公開日（9/23 JST、約 12 時間）の反響。上 3 行は Cloudflare の分析 API、下 2 行は本番 D1（9/24 00:24 時点）

項目	値
Worker の実行回数（/api/* の要求、エラー 0）	23,505
最も多かった 1 時間（16 時台）	4,364
静的ファイルの配信要求（HTML・JS・CSS、99.5% がキャッシュから）	24,673
クリアの記録が届いた端末（重複なし）	2,342
面 1-1 / 1-2 / 1-3 / 1-4 / 2-1 をクリアした端末	1,729 / 1,264 / 1,044 / 633 / 557

数値の読み方は次のとおりである。

- workers.dev のサブドメインで公開しているため、ゾーンの訪問者分析はなく、訪問者数そのものは取れない。最も近い値は、クリアの記録が届いた端末数である。
- 端末数は D1 の plays 表による。この表は公開当日の 20:44 に追加したもので、それ以前はランキング上位 100 位に入るか AI に初めて勝った走行しか残っていない。また、1 つもクリアしなかった人や、オフラインで遊んだ人は数えない。したがって 2,342 は遊んだ端末数の下限である。1 人が 2 台で遊べば 2 と数える。
- 開発中に D1 の自前カウンタで数えた値（20:31 時点で 11,754 件）は、Worker が要求数をまとめて書き込む方式のため少なめに出ていた。上の表では Cloudflare の分析 API の値を使った。

公開日の Worker 実行回数は Workers の無料枠（1 日 10 万）の約 24% で、設計段階で見積もった「1 日 2 万人が遊んでも無料枠に収まる」範囲に入っている。

7 考察

7.1 完成度を上げた工夫

記録から読み取れる工夫を、役割ごとにまとめる。

■判断を分ける

- 最初から 1 案に決めず、方向の違う 5 案を独立に作らせ、観点の違う 3 人に採点させた。重みは楽しさを最も重くした。
- 設計書を「互いに話さずに並行して作れるほど正確に」書かせ、次に「明日から作る主任エンジニア」の立場で批評させた。批評役は数値を自分で測り直し、解けない面や無料枠の超過を実装の前に見つけた。
- 指揮役はコードを書かず、各担当の報告を読んで決定事項（D1～D10、R5～R10）を出す役に徹した。

■並行させても壊れない形にする

- 契約（型）と全ファイルの仮実装を先に作り、全体が型チェックを通る状態から並列実装を始めた。
- ファイルの所有者を 1 人に決め、他人のファイルが必要なら報告に「依頼」として書かせた。git の状態を変える操作と npm install を禁じた。
- テスト用サーバーのポート番号とビルドの出力先を担当ごとに分け、同時に動いても衝突しないようにした。
- 時間のかかる計算は、仮の結果を先に出して他を待たせず、本番の計算を裏で流した。
- 台本の中で依存関係を表し、レビューは物理コアの完成を待ってから始めた。

■自己申告を信用しない

- 作る役と確かめる役を必ず分けた（土台の検証役、モジュールごとのレビュー役、総点検の反証役、CasADiの確認役）。
- 指摘には具体的な失敗の筋書きを求め、反証役には「裏付けられなければ偽とせよ」と指示した。
- 修正には回帰テストを付け、そのテストが元のコードで落ちることも確かめた。
- 「テストを弱めない」「根本原因を直す」を繰り返し指示した。
- 出力を JSON スキーマで固定し、受け入れ基準ごとに「通ったか」と「証拠」を書かせた。

■目で見える、遊ばせる

- 描画・UI・統合の担当には、スクリーンショットを撮って画像として読み、直すことを繰り返させた。
- 人の代わりに、人間らしい制約とぶれをもつボットにゲームを遊ばせ、メダルの取りやすさを数値で測った。

7.2 人が必要だったところ

公開までの人の関与は表 1 の約 10 回で、大半は公開先や公開操作に関わるものだった。その中で、ゲームの出来を最も大きく変えたのは試遊の感想（H7）である。エージェントの審査は『『ピタッ』は腕の見せ所』を重く見て自動のゆれ止めを補助扱いに下げ、ボットによる調査はキーボードの弱点を数値で示したものの、直すところまでは行かなかった。実際に遊んだ人の「減衰させようとする」という一言が、それを覆した。ボットは「画面を見ながら指で調整する感じ」までは再現できない、と指揮役自身も報告の中で述べていた。

また、外部に影響する操作（デプロイ、公開後に変えられない日付の決定）は、指揮役が必ず人に確認した。CasADi の件では、人のヒントをそのまま採用せず、検証したうえで「いまは使わない」と判断し、理由（ライセンス）を示した。

7.3 費用と限界

- 規模：開発セッションだけで 70 体、ツール呼び出し 5,921 回、約 1,900 万トークンを使った。個人の趣味の開発としては大きく、ultracode を選んだ時点でこの規模を許したことになる。
- 時間：開発セッションは約 14.4 時間で、そのうちの多くはエージェントが裏で動いている時間だった。最適化器の計算（本番ゴースト 84 分、日替わりの解き直し 2 時間 44 分）が工程の長さを決めていた。
- 規模の見積もり：前置きでは 5~15k 行としていたが、テストを含めて約 5.5 万行になった。
- 本稿の偏り：本稿の分析と執筆は Claude Code が行った。数値はすべて記録から集計したもののだが、「工夫」の選び方や評価には、作った側の視点が入りうる。
- 反響の数値：訪問者数は取れず、端末数はクリアの記録が届いたものに限られるので、表 7 の端末数は遊んだ人数の下限にすぎない。

8 まとめ

2 文の依頼から、Claude Code は 7 本のワークフローで 70 体のサブエージェントを動かし、約 14 時間で、研究の最適化器を競走相手にしたブラウザゲームを設計・実装・検証した。進め方の要点は、複数案と審査による設計、契約と仮実装の先行、所有権を分けた並列実装、作る役と確かめる役の分離、そしてスクリーンショットとボットによる「見て、遊ばせて」確かめることだった。人の関与は約 10 回で、ゲームの出来を最も大きく変えたのは、実際に遊んだ人の「難しすぎる」という一言だった。公開日の約 12 時間で、API 要求は 23,505 件、クリアの記録が届いた端末は少なくとも 2,342 台だった。

参考文献

- [1] N. Gulley, “Simulate in MATLAB, Animate in Blender,” MathWorks Blogs, 2026.
- [2] 貝淵蒼馬, 「吊り下げボールを壁の隙間に出し入れするガントリークレーン: 軌道最適化・時変 LQR・独立検証による再現」, <https://github.com/Raptor-zip/crane-ball-wall>, 2026.

付録 A 面の一覧と AI のタイム

表 8: 18 面と AI のタイム (最適化器で求めた最短時間. ゲーム内の判定で測った値)

面	名前	AI [s]	面	名前	AI [s]
1-1	おつかい	1.500	3-2	原典・ブランコ	3.858
1-2	ひざ丈の堀	2.000	3-3	短い紐	4.125
1-3	腰の高さ	2.750	4-1	ハードル走	4.375
1-4	ひろい谷間	2.142	4-2	終点	2.475
2-1	ブログの壁	2.742	4-3	たまご	3.100
2-2	原典: いれる	2.625	5-1	重い荷	2.192
2-3	原典: だす	2.650	5-2	階段ポケット	2.633
2-4	郵便受け	2.075	5-3	天井すれすれ	3.100
3-1	非力モーター	5.575	5-4	フルコース	5.775

付録 B プロンプトの抜粋

リスト 5: 5 案の方向づけ (mastery と mobile)

```
{ key: 'mastery', label: 'Mastery / beat-the-AI speedrun', brief: 'Design from the angle of a precision skill toy with deep mastery: tight, readable controls, fast retry, speedrun timers, ghost races against your best run AND against the trajectory optimiser ("AI"), medals. Think Trackmania / Getting Over It (but not cruel) / QWOP-lite physics mastery. The headline hook is "can a human beat the optimal-control AI?" },
{ key: 'mobile', label: 'Mobile-first casual + social', brief: 'Design from the angle of a phone-first, one-thumb game with 30-second sessions and strong social hooks: instantly understandable touch control, daily challenge with a shared global leaderboard (Cloudflare Worker + D1), shareable result text for X (Wordle-style), async ghost races against other players. The first 10 seconds must be self-explanatory without text.' },
```

リスト 6: 審査役「プレイヤー」の観点

You are the target player: a Japanese engineer/student who sees this on X, taps the link on a phone, and has 30 seconds of patience. Also imagine sharing it with friends. Judge on first-minute clarity, touch controls, shareability, the "wow, this is the real physics from the paper" factor, and whether you would come back tomorrow.

リスト 7: レビュー役への指示

```
# Your role: independent REVIEWER-FIXER for module ${m.key}
The owner agent just finished building this module. Its report: ${JSON.stringify(build)}
...
1. Adversarially review the module against GAME_DESIGN.md ... and against the quality bar (fun, feel, polish, robustness, performance, accessibility). Read ALL of the module's code. Look for: spec deviations, missing features, bugs, edge cases (NaN, empty data, null minGapMm, offline, rotated phone, reduced motion), non-determinism, allocation in hot loops, memory leaks, flaky tests, tests that do not really test the claim.
2. Run every test/check the module's acceptance criteria name (and add missing tests). For visual modules take screenshots and LOOK at them critically as a player on a phone would.
3. FIX everything you find directly in the module's files, re-run, and iterate until the acceptance criteria truly pass.
```

リスト 8: 難易度調査役への指示 (抜粋)

Question: are the medal thresholds (bronze $\leq 2.0 \cdot P$, silver $\leq 1.5 \cdot P$, gold $\leq 1.2 \cdot P$, crown $< P$, with $P = \text{AI parSub}$) achievable by HUMANS on each of the 18 campaign levels and on a sample of 12 daily levels?
Method: ... imports the real simulator (src/sim) and the real servo (src/input/servo.ts) and plays each level with HUMAN-LIKE controller families:

(a) touch/mouse: a target position $x_t(t)$ that is piecewise linear with ≤ 6 knots, knot spacing ≥ 0.15 s, target speed ≤ 5 m/s, then held;
(b) keyboard: velocity intents from $\{-4, 0, +4\}$ m/s (and fine ± 1) with the spec ramps, ≤ 7 switches ...
plus a human reaction/precision penalty: add Gaussian timing jitter $\sigma = 40$ ms and position jitter $\sigma = 1$ cm to the best found plan and report the success rate over 50 jittered replays.